FastAPI on AWS EC2 — Complete Deployment Guide

> Updated based on real deployment experience. Includes all gotchas and fixes encountered.

Step 1: Launch the EC2 Instance

1.1 — AMI & Instance Type

1. Go to **AWS Console → EC2 → Launch Instance**
2. Configure as follows:

Setting	Value
Name	`fastapi-poc`
AMI	**Ubuntu Server 22.04 LTS** (free tier eligible)
Instance type	`t2.micro` (free tier)
Key pair	Create new → name it `your-key` → `.pem` format → **Download immediately**

> ⚠️ **Gotcha:** Save the `.pem` file the moment AWS offers it — you cannot download it again.

1.2 — Security Group Rules

Create a new security group with these **inbound rules**:

Type	Protocol	Port	Source
SSH	TCP	22	My IP
Custom TCP	TCP	80	0.0.0.0/0

> ⚠️ **Gotcha — Office/Corporate Networks:** If you're on a corporate WiFi or VPN, the "My IP" option in AWS may not match your actual outbound IP (your machine shows a private IP like `172.x.x.x`). To find your real public IP, visit `https://checkip.amazonaws.com` in a browser and use that IP with `/32` (e.g. `203.145.12.55/32`) as the SSH source.
> If SSH still times out after setting your IP, temporarily set the SSH source to `0.0.0.0/0` to unblock yourself. Your `.pem` key still protects you from unauthorized logins, so this is acceptable for a POC.

1.3 — Storage

Keep the default **8 GiB gp2** root volume (free tier covers up to 30 GiB).

Click **Launch Instance**.

Step 2: Connect via SSH

2.1 — Fix key permissions

On Linux/macOS:

```bash
chmod 400 ~/Downloads/your-key.pem
```

On Windows (PowerShell) — note the `$(` syntax carefully:

```powershell
icacls "C:\Users\YOUR_USERNAME\Downloads\your-key.pem" /inheritance:r /grant:r "$($e
```

> ⚠️ **Gotcha:** The command uses `"$($env:USERNAME):(R)"` — not `"($env:USERNAME):(R)"`. Missing the `$` before the opening parenthesis causes a "No mapping between account names" error and the command will fail silently.

2.2 — Get your public DNS

In EC2 Console → select your instance → copy **Public IPv4 DNS**. It looks like: `ec2-XX-XX-XX-XX.eu-north-1.compute.amazonaws.com`

2.3 — Test connectivity before SSH (optional but useful)

```powershell
Test-NetConnection -ComputerName ec2-XX-XX-XX-XX.eu-north-1.compute.amazonaws.com -P
```

- `TcpTestSucceeded : True` → port is open, proceed to SSH
- `TcpTestSucceeded : False` → Security Group is still blocking you, fix inbound rules first

2.4 — SSH in

Linux/macOS:

```bash
ssh -i ~/Downloads/your-key.pem ubuntu@ec2-XX-XX-XX-XX.eu-north-1.compute.amazonaws.
```

Windows (PowerShell):

```powershell
ssh -i "C:\Users\YOUR_USERNAME\Downloads\your-key.pem" ubuntu@ec2-XX-XX-XX-XX.eu-nor
```

> ⚠️ **Gotcha:** The default user for Ubuntu AMIs is `ubuntu`, not `ec2-user` (that's Amazon Linux). Wrong username = `Permission denied` error.

Step 3: Install Python & System Dependencies

Run these on the **EC2 instance**:

```bash
# Update package index
sudo apt update && sudo apt upgrade -y
```

```bash
# Add deadsnakes PPA for Python 3.11
# Ubuntu 22.04 ships with Python 3.10 by default and doesn't have 3.11 in its defaul
sudo apt install -y software-properties-common
sudo add-apt-repository ppa:deadsnakes/ppa -y
sudo apt update
```

```bash
# Install Python 3.11, venv, pip, and build tools
sudo apt install -y python3.11 python3.11-venv python3-pip build-essential libsqlite
```

> ⚠️ **Gotcha:** Skipping `build-essential` and `libsqlite3-dev` causes ChromaDB to fail during installation with cryptic native compilation errors.

Verify:

```bash
python3.11 --version
pip3 --version
```

Step 4: Clone Your Project via Git

```bash
# Navigate to home directory
cd ~/

# Clone your repository
git clone https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git

# Verify
ls ~/
```

> ✅ `git` is pre-installed on Ubuntu 22.04 — no separate installation needed.

Step 5: Set Up Python Virtual Environment

```bash
# Navigate into your project folder
cd ~/YOUR_REPO_NAME

# Create virtual environment
python3.11 -m venv venv
```

> No output = success on Linux.

```bash
# Activate virtual environment
source venv/bin/activate
```

You'll see `(venv)` appear at the start of your terminal prompt — this confirms it's active.

Step 6: Install Python Dependencies

```bash
# Upgrade pip first to avoid legacy resolver issues
pip install --upgrade pip

# Install dependencies
pip install -r requirements.txt
```

> ⚠️ **Gotcha — ChromaDB:** ChromaDB compiles native code and can take 3–5 minutes. Don't interrupt it. If it fails, retry with:
>
> ```bash
> pip install chromadb --no-cache-dir
> ```

Step 7: Set Up the `.env` File

```bash
# Make sure you're in your project directory
cd ~/YOUR_REPO_NAME

# Create the .env file
nano .env
```

Paste your environment variables:

```
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxx
```

Save: `Ctrl+O` → `Enter` → `Ctrl+X`

Lock down file permissions:

```bash
chmod 600 .env
```

Verify contents:

```bash
```

```
cat .env
```

> ⚠️ **Gotcha — Typos in variable names:** Make sure the variable name matches exactly what your app expects. For example `OPENAI_API_KEY` is NOT the same as `OPEN_API_KEY`. A typo here causes a `RuntimeError` at startup that can be confusing to debug since the file exists but the key isn't found.

Step 8: Configure the systemd Service

8.1 — Find your FastAPI entry point

Before creating the service, confirm:

1. What is your main Python file? (e.g. `main.py`)
2. What is the FastAPI instance variable? Check with:

bash

```
head -20 ~/YOUR_REPO_NAME/main.py
```

Look for a line like `app = FastAPI()` — the variable name (e.g. `app`) goes into the `ExecStart` line as `main:app`.

8.2 — Create the service file

bash

```
sudo nano /etc/systemd/system/fastapi.service
```

Paste this (replace `YOUR_REPO_NAME` and `main:app` as needed):

ini

```
[Unit]
Description=FastAPI Application
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/YOUR_REPO_NAME
Environment="PATH=/home/ubuntu/YOUR_REPO_NAME/venv/bin"
EnvironmentFile=/home/ubuntu/YOUR_REPO_NAME/.env
ExecStart=/home/ubuntu/YOUR_REPO_NAME/venv/bin/uvicorn main:app --host 0.0.0.0 --por
Restart=always
RestartSec=5
AmbientCapabilities=CAP_NET_BIND_SERVICE
CapabilityBoundingSet=CAP_NET_BIND_SERVICE

[Install]
WantedBy=multi-user.target
```

Save: `Ctrl+O` → `Enter` → `Ctrl+X`

> **Key details explained:**
>
> - `EnvironmentFile` — loads your `.env` automatically, your app doesn't need extra code to find it
> - `AmbientCapabilities=CAP_NET_BIND_SERVICE` — lets a non-root process bind to port 80 cleanly
> - `Restart=always` + `RestartSec=5` — auto-restarts on crash with a 5s delay
> - `After=network.target` — waits for network before starting on reboot

Step 9: Start and Verify the Service

Run these one at a time:

```bash
```

```
# Reload systemd to pick up the new service file
sudo systemctl daemon-reload

# Enable auto-start on every reboot
sudo systemctl enable fastapi

# Start the service now
sudo systemctl start fastapi

# Check status
sudo systemctl status fastapi
```

Healthy output looks like:

```
Active: active (running)
...
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:80
```

If you see `activating (auto-restart)` or `exit-code`, check logs:

```
bash

sudo journalctl -u fastapi -n 50
```

Common errors and fixes:

Error in logs	Fix
`OPENAI_API_KEY is not set`	Check your `.env` for typos in variable name
`ModuleNotFoundError`	Wrong `ExecStart` path or venv not pointing correctly
`Failed to bind to port 80`	`AmbientCapabilities` lines missing or misspelled in service file
`.env not found`	Double-check `EnvironmentFile` path matches actual `.env` location

Open in your browser:

```
http://ec2-XX-XX-XX-XX.eu-north-1.compute.amazonaws.com/docs
```

You should see the **Swagger UI** with all your endpoints.

Or test via curl:

```bash
curl http://ec2-XX-XX-XX-XX.eu-north-1.compute.amazonaws.com/
```

> ⚠️ **Gotcha:** If the browser shows "This site can't be reached", check your Security Group inbound rules allow port **80** from `0.0.0.0/0`.

Quick Reference Cheatsheet

```bash
# Service management
sudo systemctl start fastapi      # Start
sudo systemctl stop fastapi       # Stop
sudo systemctl restart fastapi    # Restart after code changes
sudo systemctl status fastapi     # Check status

# Logs
sudo journalctl -u fastapi -f     # Live logs (Ctrl+C to exit)
sudo journalctl -u fastapi -n 50  # Last 50 lines

# After updating code (pull latest + restart)
cd ~/YOUR_REPO_NAME
git pull
sudo systemctl restart fastapi
```